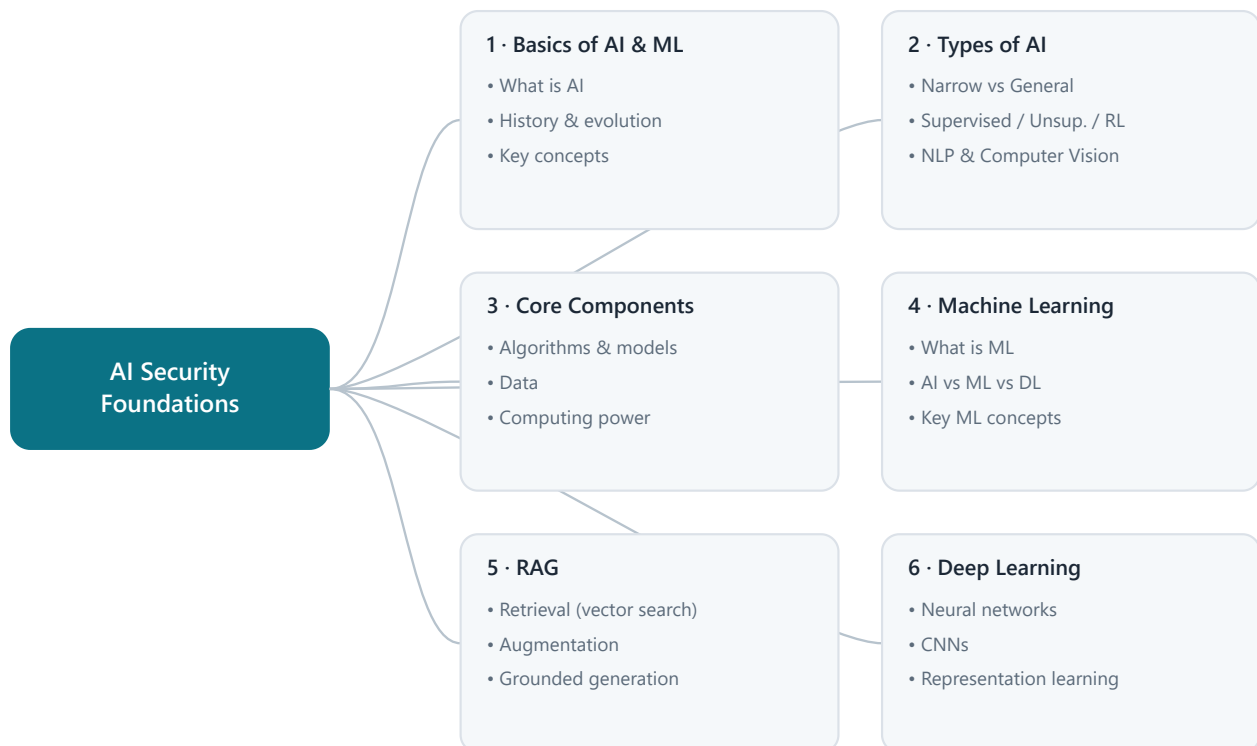


# An Overview of AI Security — Foundations

The AI/ML fundamentals you need before AI security proper. Every concept is explained on its own terms, then re-examined through a **Security Lens** that connects it to threat modeling, detection, and defense. All diagrams are drawn inline — they render anywhere, no plugins.

## Contents

- |                                           |                                         |
|-------------------------------------------|-----------------------------------------|
| <b>1</b> Basics of AI and ML              | <b>5</b> Retrieval Augmented Generation |
| <b>2</b> Types of AI                      | <b>6</b> Basics of Deep Learning        |
| <b>3</b> Core Components of AI Systems    | <b>A</b> AI Security Attack Taxonomy    |
| <b>4</b> Introduction to Machine Learning | <b>B</b> Governing Frameworks           |



## Section 1

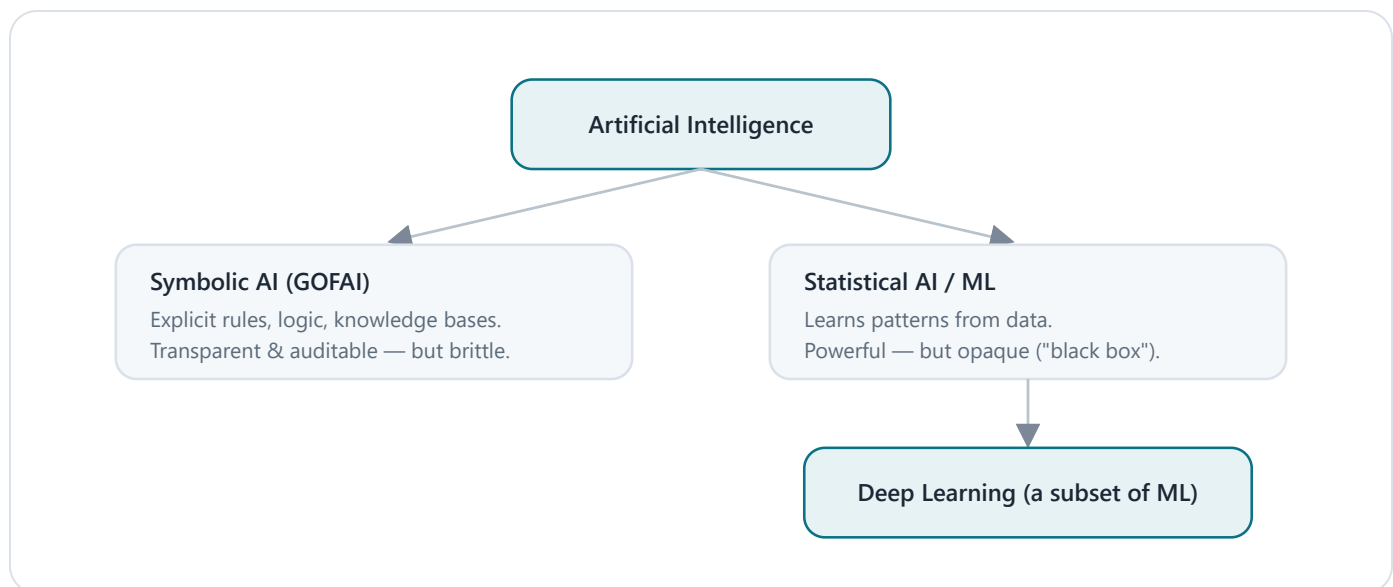
# Basics of AI and ML

## 1.1 · What is AI?

**Artificial Intelligence (AI)** is the field of building systems that perform tasks normally requiring human intelligence — perceiving, reasoning, learning, planning, understanding language, and making decisions.

Most modern engineering uses the **rational agent** view: an agent perceives its environment through sensors and acts through actuators to maximize its expected performance. Two axes help frame "intelligence": *thinking vs. acting* (does it reason like a human, or just behave usefully?) and *human-like vs. rational* (does it imitate humans, or act optimally toward a goal?).

### The two great families of AI



Symbolic systems encode logic you can read; learned systems hide their logic in numeric weights.

## Security Lens — why the definition matters

A symbolic system's logic can be reviewed like source code. A learned model's "logic" lives in millions/billions of numeric weights that cannot be directly inspected. This opacity is the root cause of most AI-specific risk: you often can't tell *why* a model decided something, whether it memorized sensitive data, or whether an attacker nudged it. Your threat model shifts from "review the code" to "reason about data, training, and inference behavior."

## 1.2 · History and Evolution of AI



Amber dots mark the two AI winters. Each boom introduced new capability — and a new attack surface.

The story to be able to tell: philosophical foundations (1943–56) named the field at Dartmouth; early optimism around the Perceptron cooled when its limits (the XOR problem) were exposed, triggering the **first winter**; rule-based **expert systems** overpromised into a **second winter**; the field then shifted from hand-coded rules to *learning from data*; **AlexNet (2012)** proved deep networks + big data + GPUs was the winning formula; and the **Transformer (2017)** unlocked the LLM/generative wave.

#### 🔒 **Security Lens — the tech always outran its defenses**

Expert systems introduced tamperable rule bases; statistical ML introduced data poisoning and adversarial inputs; LLMs/RAG introduced prompt injection and training-data extraction. Security has trailed capability at every step — you're catching up on a decade of accumulated, under-defended surface.

### 1.3 · Key Concepts in AI

- **Agent & environment** — anything that perceives (sensors) and acts (actuators). Described with **PEAS**: Performance measure, Environment, Actuators, Sensors.
- **Rationality** — acting to maximize *expected* performance given what the agent knows (not omniscience).
- **Search & optimization** — exploring possible states/solutions efficiently, often with heuristics.
- **Knowledge representation & reasoning** — how facts are stored and how new conclusions are inferred.
- **Learning** — improving performance from data/experience (the ML core).
- **Uncertainty** — real-world agents reason probabilistically (Bayesian methods).

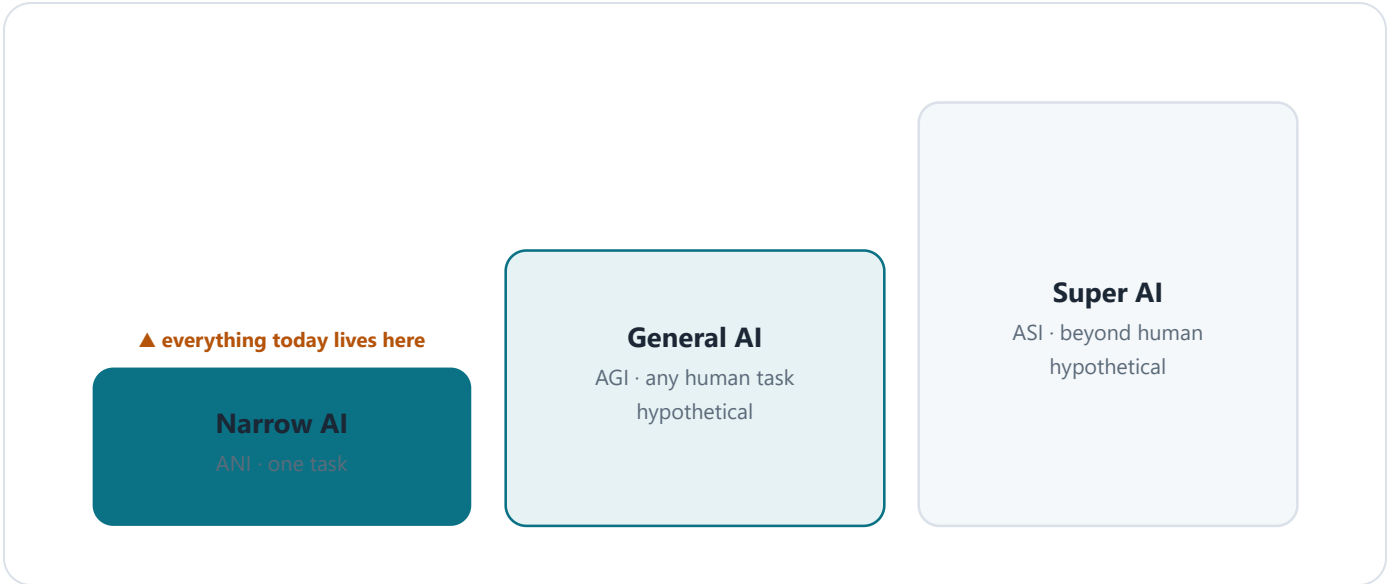
---

## Section 2

# Types of AI

There are **two orthogonal** ways to classify AI. Don't confuse them: one is about *how general* the system is; the other is about *how it learns*.

2.1 · Narrow AI vs. General AI



Capability tiers. Even the strongest LLM today is Narrow AI — broad within language tasks, still task-bounded.

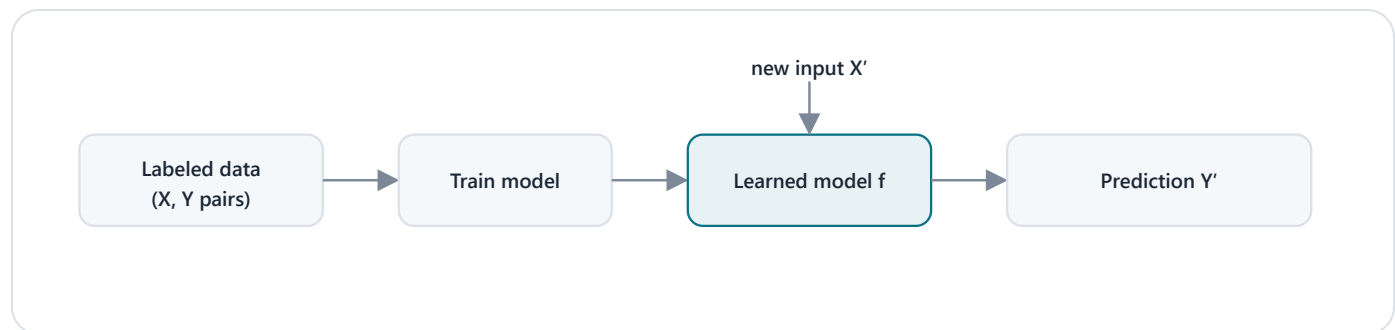
|          | Narrow AI (ANI)                                   | General AI (AGI)             | Super AI (ASI)           |
|----------|---------------------------------------------------|------------------------------|--------------------------|
| Scope    | One specific task/domain                          | Any task a human can do      | Surpasses humans broadly |
| Status   | Everything that exists today                      | Hypothetical / research goal | Hypothetical             |
| Examples | Spam filters, image classifiers, ChatGPT, AlphaGo | —                            | —                        |

 Security Lens

Nearly all operational risk is *narrow-AI risk, here and now* — poisoning, evasion, prompt injection, data leakage. Keep threat modeling grounded in deployed systems; AGI-alignment debates are a different problem class than defending this quarter's production model.

2.2 · Supervised Learning

Learn a mapping from inputs **X** to outputs **Y** using **labeled** examples, then generalize to unseen inputs.



| Sub-type              | Output           | Example tasks                                  | Algorithms                                                                     |
|-----------------------|------------------|------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Classification</b> | Discrete class   | Spam vs. not-spam, malware family, image label | Logistic regression, SVM, trees, random forest, gradient boosting, neural nets |
| <b>Regression</b>     | Continuous value | Price, risk score, latency                     | Linear regression, regression trees, neural nets                               |

#### **Security Lens**

Supervised learning depends entirely on **label integrity**. Corrupt the labels or inject mislabeled samples (**data poisoning**) and the model learns the attacker's intended behavior — e.g., a poisoned training set teaching a malware classifier that a specific attacker tool is "benign."

## 2.3 · Unsupervised Learning

Find structure in **unlabeled** data — patterns, groupings, or compressed representations discovered on its own.

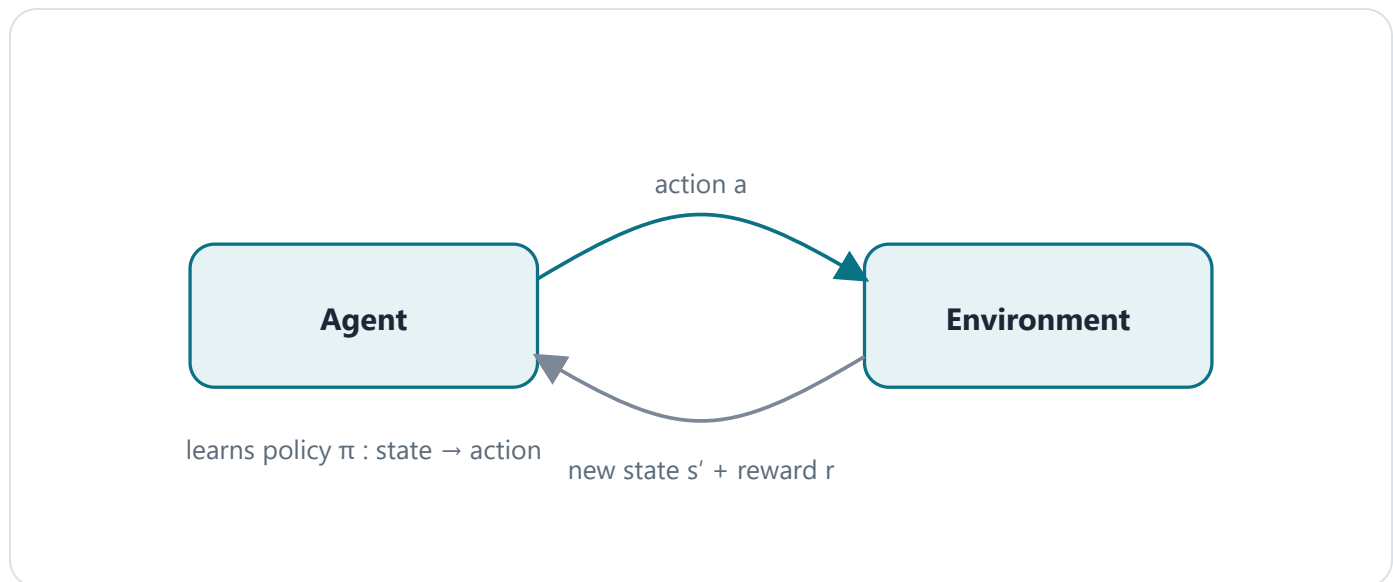
- **Clustering** — group similar items (k-means, hierarchical, DBSCAN). Log clustering, user segments.
- **Dimensionality reduction** — compress many features into a few (PCA, t-SNE/UMAP, autoencoders).
- **Anomaly detection** — flag data points that don't fit learned "normal."

### Security Lens — your home turf

Unsupervised **anomaly detection** underpins UEBA, beaconing detection, and endpoint/DNS outlier hunting. Two things to internalize: (1) "normal" is defined by training data, so **baseline poisoning** lets an attacker slowly drift the baseline until malicious activity looks normal (boiling-frog); (2) the real defensive challenge is *precision*, not just recall — low precision buries the SOC.

## 2.4 · Reinforcement Learning (RL)

An **agent** learns by **interacting** with an **environment**, taking **actions**, and receiving **rewards**. The goal is a **policy** that maximizes cumulative long-term reward — learning by trial and error, not from a fixed labeled set. Formalized as a **Markov Decision Process (MDP)**.



The core loop. RLHF — the technique used to align LLMs to human preferences — is built on this.

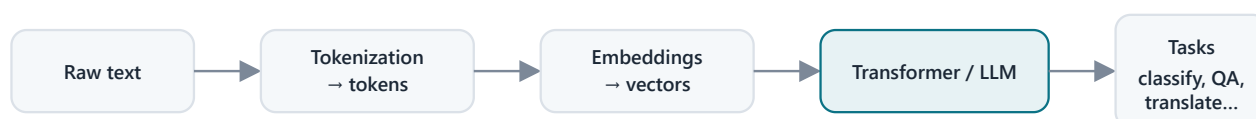
The central tension is **exploration vs. exploitation**: try new actions to find better rewards, or exploit known-good ones. You'll meet RL in game-playing (AlphaGo), robotics, control, and **RLHF** for LLMs.

### Security Lens

**Reward hacking:** an agent exploits a poorly specified reward to score high without the intended behavior. For LLMs, manipulating the feedback/reward signal (or the labelers behind it) is an alignment attack surface. Agentic AI that *takes actions* in real systems can be steered into harmful actions if its objective or observations are tampered with.

## 2.5 • Natural Language Processing (NLP)

Enabling machines to understand and generate human language.



Evolution: rule-based → statistical (n-grams) → RNN/LSTM → Transformers → LLMs. Self-attention weighs every token against every other, which is why it scales.

### Security Lens

NLP/LLM systems are where **prompt injection** lives — the top LLM-specific risk. Because these systems don't cleanly separate *instructions* from *data*, attacker-controlled text (a user message, a web page, a retrieved document, an email) can hijack behavior. Also relevant: **sensitive-information disclosure** and **jailbreaks**.

## 2.6 • Computer Vision (CV)

Enabling machines to interpret visual data.

| Task                 | Output                             |
|----------------------|------------------------------------|
| Image classification | One label for the whole image      |
| Object detection     | Bounding boxes + labels per object |



| Task                  | Output                            |
|-----------------------|-----------------------------------|
| Semantic segmentation | A class label for every pixel     |
| Instance segmentation | Pixel masks per individual object |

Historically powered by **CNNs** (Section 6.3); increasingly by **Vision Transformers (ViT)**.

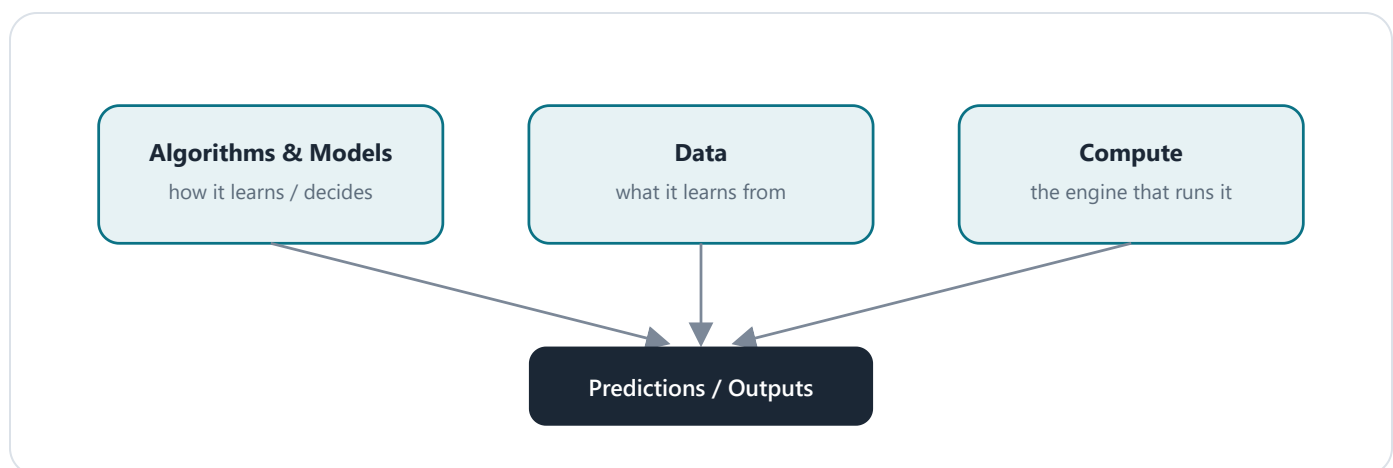
#### Security Lens

CV is the classic stage for **adversarial examples**: a few crafted pixels (imperceptible to humans) flip a classifier's output. Physical variants — a sticker on a stop sign — matter for camera/vision products. Deepfakes and synthetic media (a generative-CV problem) feed phishing, fraud, and identity attacks.

## Section 3

# Core Components of AI Systems

Every AI system rests on three interdependent pillars. Weakness in any one degrades the whole — and each is a distinct attack surface.



## 3.1 • Algorithms and Models

Keep these distinct — people conflate them constantly:

- **Algorithm** = the *procedure/recipe* for learning (gradient descent, decision-tree induction, backpropagation). Fixed and general.
- **Model** = the *learned artifact* produced by running an algorithm on data. A model = **architecture** (structure) + **parameters/weights** (learned numbers).

Algorithm + Data + Compute  $\xrightarrow{\text{train}}$  Model (architecture + weights)  
Model + New input  $\xrightarrow{\text{infer}}$  Prediction

### Security Lens

The trained model is an asset to protect. Threats: **model theft/extraction** (query an API enough to clone it), **model tampering** (alter weights, embed a **backdoor**), and **supply-chain risk** — a pretrained model or checkpoint pulled from a public hub can be maliciously modified/trojaned. Treat model artifacts like signed binaries: provenance, integrity checks, a vetted registry.

## 3.2 · Data

Data is the fuel, and **data quality dominates model quality** ("garbage in, garbage out").

| Dimension          | Why it matters                                               |
|--------------------|--------------------------------------------------------------|
| Quantity           | Deep models need large datasets to generalize                |
| Quality            | Errors, noise, and mislabels propagate into the model        |
| Representativeness | Unrepresentative data → bias and poor real-world performance |
| Labeling           | Supervised learning depends on correct labels                |
| Splits             | Train / validation / test must not leak into each other      |
| Drift              | Real-world data shifts over time → model decay               |

#### Security Lens

Data is the richest attack surface in ML: **training-data poisoning**, **label flipping**, **backdoor triggers** (a hidden pattern forcing a target output), and **privacy attacks** (reconstruct data or infer membership from a trained model). Provenance, integrity, access control, and lineage are AI-security controls — not just governance nice-to-haves.

### 3.3 · Computing Power

- **Hardware** — GPUs/TPUs provide the parallelism deep learning needs; CPUs orchestrate.
- **Training vs. inference** — training is expensive and periodic; inference is cheaper per call but runs constantly at scale.
- **Distributed training** — large models split across many accelerators.
- **Scaling trend** — capability has grown roughly with more data + more parameters + more compute.

#### Security Lens

Compute is both a DoS target and a supply-chain concern. LLM apps face **unbounded resource consumption** ("denial of wallet" — driving up inference cost) and model DoS via expensive queries. The training infra, GPU clusters, MLOps tooling, and dependencies form a software supply chain needing the same hygiene as any CI/CD system.

---

## Section 4

# Introduction to Machine Learning

### 4.1 · What is Machine Learning?

**Machine Learning (ML)** is the subset of AI where systems **learn patterns from data** to improve at a task, without being explicitly programmed for every rule. A common

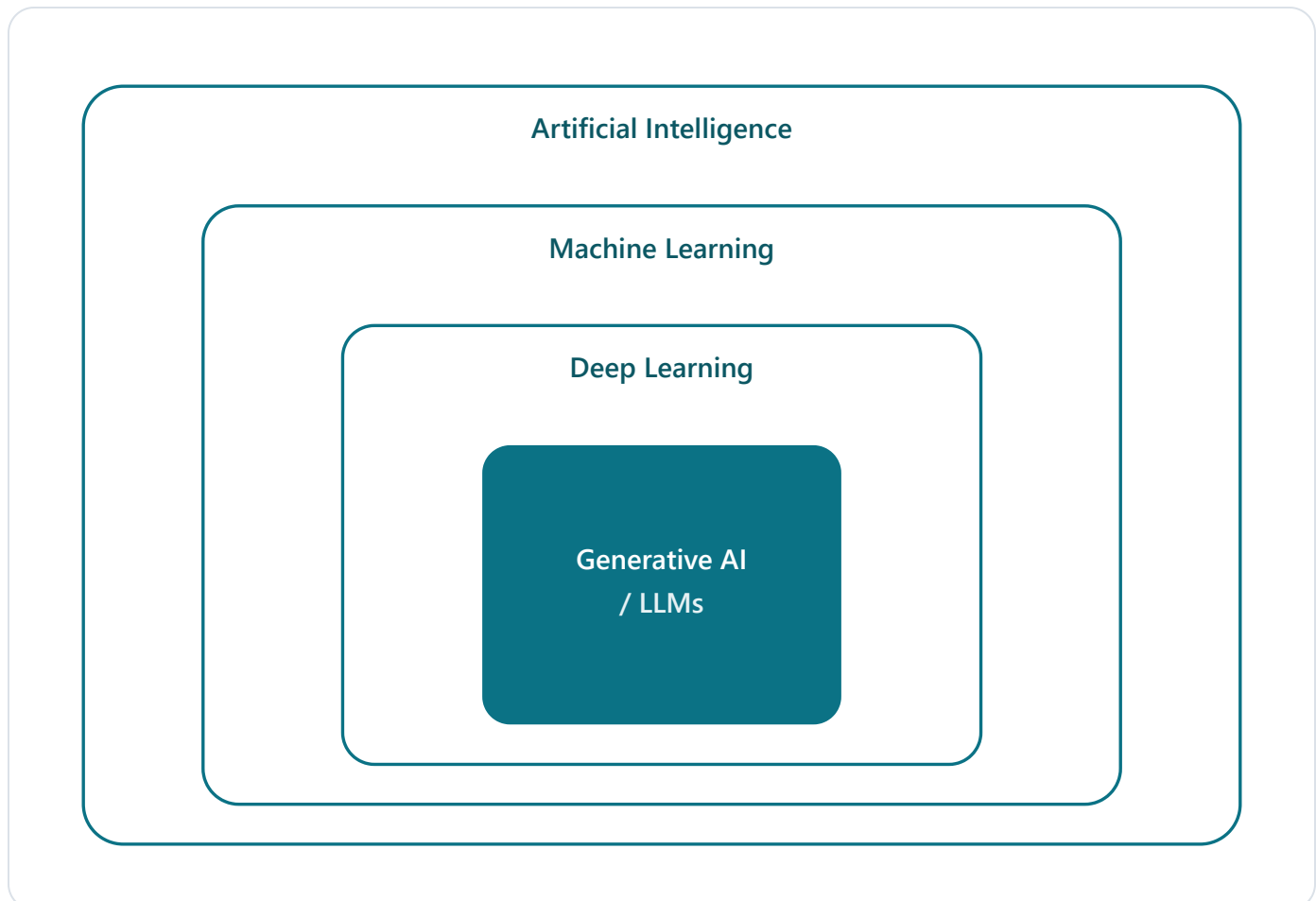
working definition (paraphrasing Tom Mitchell): *a program learns from experience  $E$  on a task  $T$ , measured by performance  $P$ , if its performance on  $T$  improves with  $E$* . The term is usually credited to **Arthur Samuel (1959)**.



Traditional code: humans write the rules. ML: the machine derives rules from data + examples.

## 4.2 · Differences between AI, ML, and DL

These are **nested subsets**, not competitors.



|                            | AI                                     | ML                                 | Deep Learning                       |
|----------------------------|----------------------------------------|------------------------------------|-------------------------------------|
| <b>Scope</b>               | Broadest — includes rule-based systems | Subset of AI that learns from data | Subset of ML using deep neural nets |
| <b>Feature engineering</b> | N/A or manual                          | Often manual                       | Automatic (learned representations) |
| <b>Data appetite</b>       | Varies                                 | Moderate                           | Large                               |
| <b>Interpretability</b>    | High (symbolic)                        | Moderate                           | Low (opaque)                        |

### 4.3 · Key ML Concepts

- **Features & labels** — features are input variables; the label is the target you predict.
- **Train / validation / test split** — train to *learn*, validate to *tune*, test to *honestly measure*. Test data stays unseen until the end.
- **Overfitting vs. underfitting** — overfitting memorizes training data (incl. noise) and fails on new data (high variance); underfitting is too simple to capture the pattern (high bias). The **bias–variance tradeoff** balances them.
- **Parameters vs. hyperparameters** — parameters are learned (weights); hyperparameters are chosen by you (learning rate, tree depth, layer count).
- **Loss + gradient descent** — loss quantifies "how wrong"; gradient descent iteratively adjusts parameters to reduce it.
- **Regularization** — penalize complexity to curb overfitting.

#### Precision vs. recall — essential for security work

|                          | Predicted Positive  | Predicted Negative  |
|--------------------------|---------------------|---------------------|
| <b>Actually Positive</b> | True Positive (TP)  | False Negative (FN) |
| <b>Actually Negative</b> | False Positive (FP) | True Negative (TN)  |

**Precision** =  $TP / (TP + FP)$  — *of what I flagged, how much was real?* (controls alert fatigue)

**Recall** =  $TP / (TP + FN)$  — *of all real threats, how many did I catch?* (controls misses)

**F1** = harmonic mean of precision and recall.

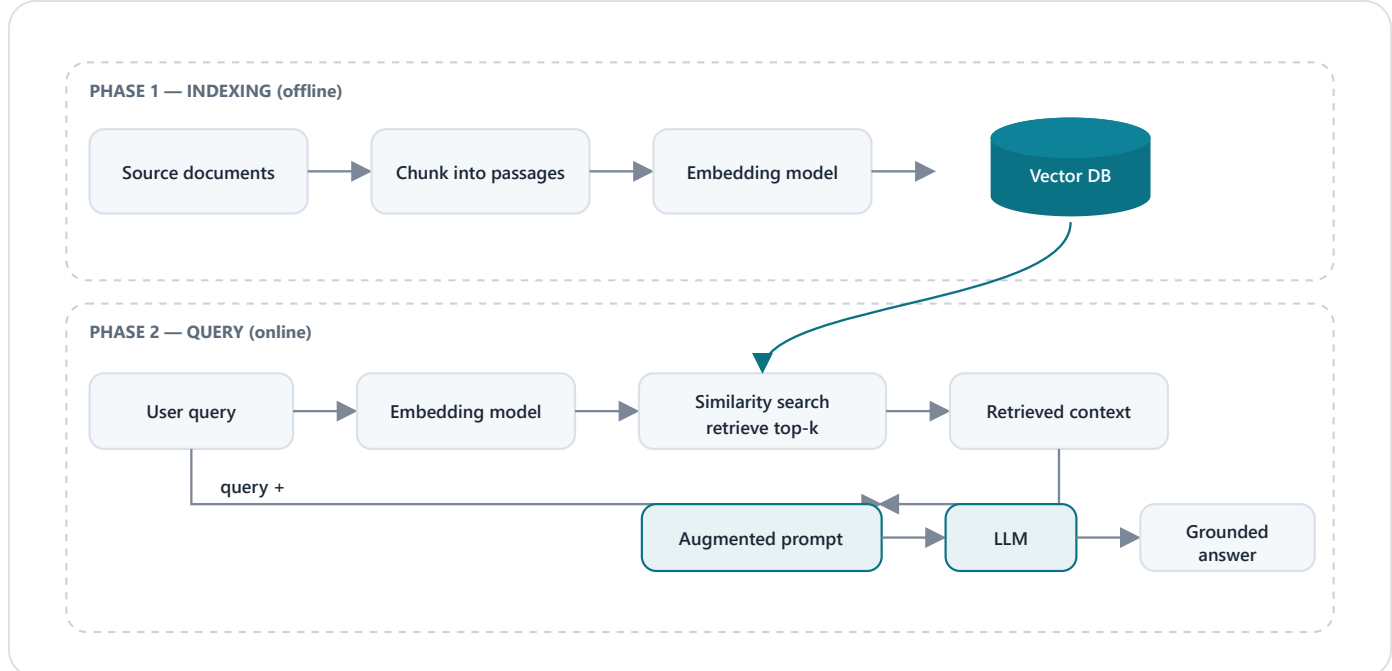
### Security Lens

In detection engineering, the precision/recall balance *is* the product. High recall + low precision buries the SOC; high precision + low recall misses intrusions. Overfitting has a security twist too: an overfit model **memorizes training data**, enabling **membership inference** and **data-extraction** — a privacy leak, not just an accuracy problem.

## Section 5

# Retrieval Augmented Generation (RAG)

An LLM's knowledge is **frozen** at training time; on its own it can't access private/current data and may **hallucinate**. **RAG** retrieves relevant external information at query time and feeds it into the model's context so the answer is **grounded** — without retraining.



Offline indexing builds the vector store; at query time, retrieved chunks are injected into the prompt before generation.

**Step by step:** ingest & **chunk** documents → **embed** each chunk into a vector (similar meaning → nearby vectors) → **store** in a **vector database** (FAISS, Pinecone, Milvus, Weaviate, Chroma, pgvector) → at query time **retrieve** the top-k most similar chunks → **augment** the prompt → **generate** a grounded answer, ideally with citations.  
Benefits: current/private knowledge without retraining, less hallucination, source attribution, cheap updates.

### **Security Lens — RAG concentrates several LLM risks**

- **Indirect prompt injection** — retrieved documents become part of the prompt, so an attacker who can plant text in a source (wiki, shared doc, crawled site) can smuggle instructions. Instruction and data share one channel.
- **Knowledge-base / retrieval poisoning** — corrupt the indexed content and you corrupt every downstream answer.
- **Broken access control** — if retrieval doesn't enforce the *user's* permissions, RAG becomes a confused deputy leaking documents. Authorize retrieval per identity, not just at the app boundary.
- **Embedding weaknesses** — embeddings can leak source text (inversion); poorly isolated stores risk cross-tenant leakage.

Treat retrieved content as **untrusted input**: isolate instructions from data, enforce least privilege on the index, validate and attribute outputs.

---

## Section 6

# Basics of Deep Learning

## 6.1 · What is Deep Learning?

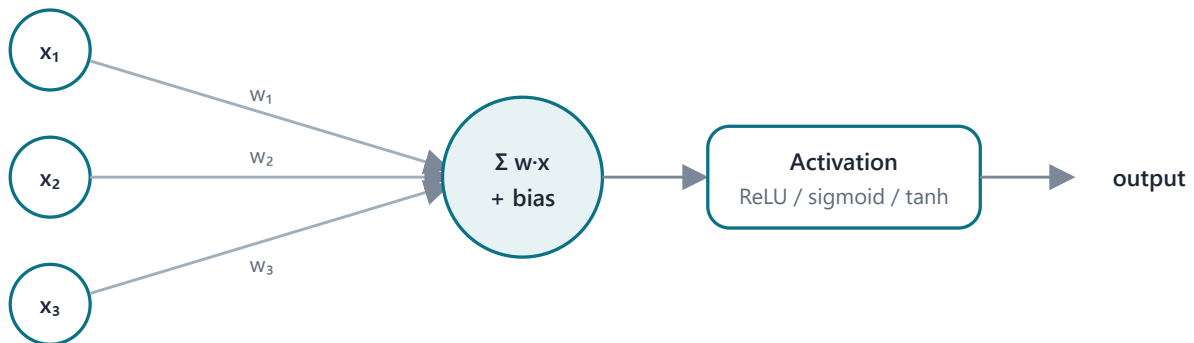
**Deep Learning (DL)** is the subset of ML that uses **neural networks with many layers**. Its superpower is **representation learning**: instead of humans hand-crafting features, the network **learns hierarchical features automatically** — simple patterns in early layers combining into complex concepts deeper in. The cost: it's data-hungry, compute-hungry, and opaque.

## 6.2 · Introduction to Neural Networks

### The single neuron

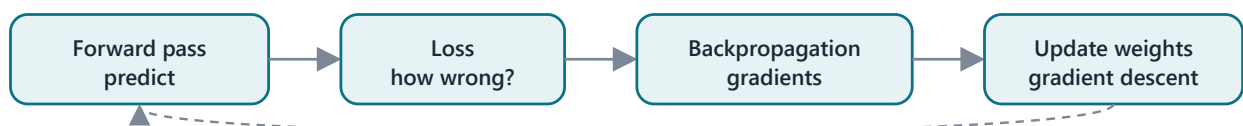
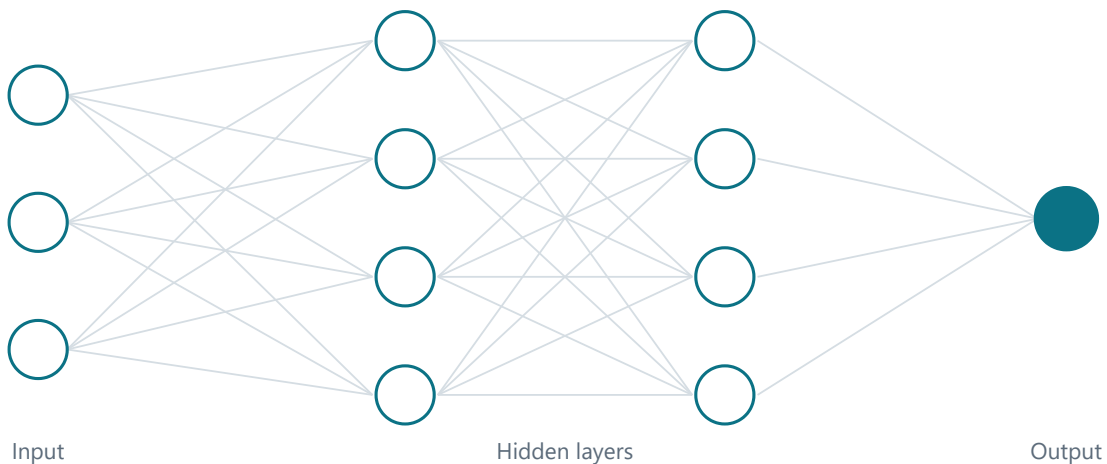


Each neuron multiplies inputs by **weights**, adds a **bias**, then passes the sum through a non-linear **activation function**.



output = activation(  $\sum(w_i \cdot x_i) + \text{bias}$  ). Activations add the non-linearity that lets stacked layers learn complex functions.

## The network & how it learns



Repeated over many epochs. Common activations: ReLU (hidden layers), sigmoid/softmax (outputs), tanh.

## Security Lens

The learned decision boundary is high-dimensional and non-intuitive — which is exactly why **adversarial examples** work. Attackers use the model's own gradients (e.g., FGSM/PGD) to compute tiny perturbations that push an input across the boundary while looking unchanged. White-box (gradient) access makes this easy; black-box attacks work via queries. Defenses (adversarial training, input validation, monitoring) help but none are complete.

## 6.3 • Convolutional Neural Networks (CNNs)

**CNNs** are neural networks specialized for **grid-like data**, above all images. They exploit spatial structure instead of wiring every pixel to every neuron.



**Three signature operations: convolution** (small filters/kernels slide over the image producing feature maps — with *parameter sharing* and *translation invariance*), **pooling** (downsamples, keeping strongest signals and adding robustness to shifts), and **fully connected layers** (produce the final classification).



Landmark CNNs: LeNet-5 (1998), AlexNet (2012), VGG, ResNet.

## Security Lens

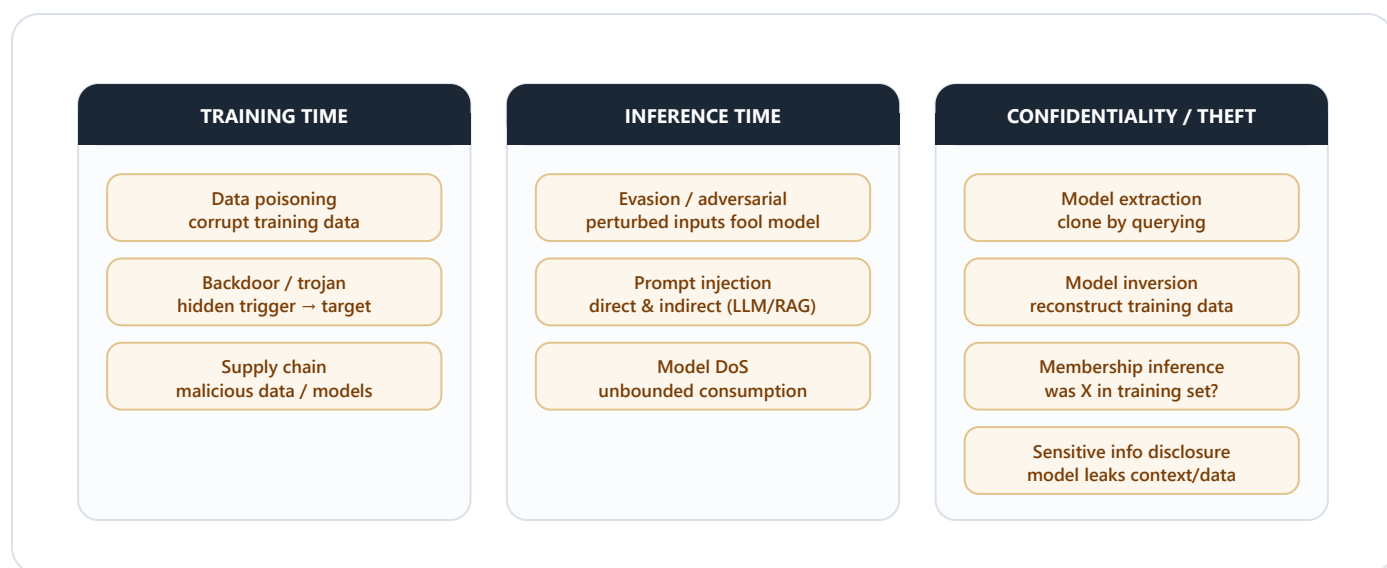
- **Adversarial patches/perturbations** — physical stickers or patterns that fool detectors (e.g., misclassifying a stop sign).
- **Backdoor/trojan attacks** — a CNN trained on poisoned data behaves normally until it sees a hidden trigger, then outputs the attacker's target. Especially dangerous with pretrained models from untrusted sources.
- **Model extraction & inversion** — cloning a deployed vision model or reconstructing training images.

Provenance of both **data and pretrained weights** is the strongest early control.

## Appendix A

# AI Security Attack Taxonomy

A compact map of the AI-specific threats referenced throughout, organized by *when* in the lifecycle they strike.



## Appendix B

# Governing Frameworks

Use these as the control catalogs and shared vocabulary of AI security. Standards evolve — confirm the current version/list on the official source before citing specific item numbers.

- **NIST AI Risk Management Framework (AI RMF 1.0)** — organizes AI risk around four functions: **Govern, Map, Measure, Manage**. Best for program-level structure.
- **MITRE ATLAS** (*Adversarial Threat Landscape for AI Systems*) — an ATT&CK-style knowledge base of real-world adversarial-ML tactics and techniques. Best for threat modeling and detection mapping.
- **OWASP Top 10 for LLM Applications** — the go-to list of LLM-specific risks, including prompt injection, sensitive information disclosure, supply chain, data & model poisoning, improper output handling, excessive agency, and vector/embedding weaknesses. Best for application-level review.

---

Natural next steps: (1) deep-dive adversarial ML with MITRE ATLAS case studies, (2) hands-on prompt-injection and RAG hardening, (3) securing the ML/MLOps pipeline as a software supply chain.